

EEE3096S Practical 1B

Execution Timing and Cycle-Accurate Assembly on the UCT Dev Board

Department of Electrical Engineering
University of Cape Town

2026

1 Overview

This practical tests your skill at measuring, controlling and proving the execution timing of software on the UCT Dev Board. You work in C and in Assembly. You measure with an on-chip timer and you verify every measurement with a two-channel oscilloscope. By the end of this practical you are able to:

1. Build a golden measure and verify it against hand-checked values.
2. Implement a fast algorithm on the board and verify its outputs against the golden values.
3. Measure execution time with a free-running 16-bit timer, including counter wrap-around.
4. Confirm timing with the oscilloscope and explain every difference between prediction and measurement.
5. Quantify the effect of the optimiser flags on execution time and code size.
6. Write Assembly with a documented cycle budget and hit a stated timing target within scope reading tolerance.
7. Drive a peripheral by bit-banging and diagnose an analog timing fault on a level-shifted signal line.
8. Calculate a circuit parameter from a measured waveform and use the result to size a software delay.
9. Read the reference manual and the datasheets for the numbers you need, and name the sections you used.

Equipment and Software:

- UCT Dev Board with the STM32F051C8 and the WC1602A character LCD fitted.
- Two-channel digital oscilloscope with time cursors and screen capture or USB export.
- Laboratory signal generator with sine output, amplitude control, and DC offset control.
- PC with STM32CubeIDE v1.19, or a bare GNU ARM toolchain with `make`, `GDB`, `objdump`, and the `size` utility.
- Serial terminal for the ST-Link V2.1 virtual COM port.
- Digital multimeter and bench power supply for supply checks at each bench.

Reference documents: RM0091 reference manual, STM32F051x4/x6/x8 datasheet, HD44780 LCD controller datasheet, and the UCT Dev Board schematic and pinout reference.

Marks come from two components. The report carries the written and photographic evidence. A live demonstration in the White Lab carries the behaviour of your implementation. Submit no video files. See Section 4 for the evidence rules.

2 Tasks

Complete the tasks in order. Each task depends on a verified result from the previous task. The task text names the result. Your numbers come from your own board and your own build. The numbers differ between boards and between builds. The follow-up question at the end of each task names a result from your own run. Answer it from your own records and include the answer in the report.

Task 1: The Golden Measure on the PC

Write a C program on the PC computing the integer square root of any 32-bit unsigned input x , the largest integer whose square does not exceed x . Write the golden measure version using double precision arithmetic and the standard library square root. Produce the golden output for each input in the list: 0, 1, 15, 16, 4095, 65535, 123456789, 987654321, 4294836225, 4294967295. Verify every output by hand: square the result, confirm it does not exceed the input, and confirm the square of the next integer does. Time one call of the golden measure with `clock_gettime` on the PC.

Before you run the timing, write down your predicted time per call in nanoseconds. Base the prediction on an instruction count estimate and the CPU clock of your PC. Record the prediction and the reasoning.

Run the timing over a loop of many repetitions. Run the loop twice with different repetition counts and confirm the time per call stays stable.

The predicted time per call and your reasoning. The number of repetitions in each run. The measured time per call from each run. The mean and the spread across runs. The output table for the ten inputs. The hand-check for at least one input, written out in full.

Question. State the golden output for the input 987654321 and the time per call you measured in nanoseconds. State the number of repetitions you used. Explain why one call timed alone does not give a reliable time.

Task 2: The Fast Version on the Board

The golden table from Task 1 is your reference. Without it your firmware has no standard to test against.

Implement the same integer square root as a fast C function on the board. The function must return the correct integer square root for any input from 0 to 4294967295. Your firmware verifies the function against the ten golden values from Task 1 and indicates pass or fail. Measure the execution time of one call. Choose a timer with a 16-bit counter and run the counter freely. Record the counter value before the call and after the call. Convert the difference to time with the counter clock you configured. Handle the counter wrap-around correctly. Toggle a GPIO pin low before the timed section and high after it, so the oscilloscope shows the execution window on the same pin.

Before you run any code, write down your predicted count span and your predicted time for one call in microseconds. Base the prediction on an instruction count estimate and the cycle time of your system clock. Record both numbers.

Measure the pulse on the oscilloscope. Your counter-derived time and the scope pulse must agree within the reading tolerance of the scope. For the wrap case, time a run long enough to cross the counter overflow more than once, with the same timing arithmetic, and confirm the mean time per call agrees with the single-call time.

Lookup. Consult the RM0091 reference manual: Section 6 for the clock tree from HSI to the timer, and the general-purpose timer chapters for the counter register. Name the sections you used in your records.

The clock path you configured, naming each divider. The predicted count span and time. The measured count span. The scope pulse width. The full calculation from the span to the time. The long-run total, the wrap arithmetic, and the mean time per call. A sketch or photograph of the pulse with cursor values.

Question. State the scope pulse width you measured for one call in microseconds and the count span

you recorded. Show the full calculation from your divider values to the pulse width. State whether your system clock matches the 8 MHz HSI and how you confirmed it.

Task 3: Optimisation Flags

This task reuses the timing setup from Task 2 and its measured single-call time as the baseline.

Build the same code at four optimisation levels: `-O0`, `-O1`, `-O2`, `-Os`. Run the same timed measurement with the same setup at each level. Record the execution time per call at each level. Record the size of the program text at each level from the linker map or the size utility. Inspect the disassembly of your square root function at `-O2` and identify one optimisation transformation the compiler applied.

Before you build the `-O2` version, write down the ratio you predict between the `-O0` time and the `-O2` time.

Measure the pulse at every level with the oscilloscope. The counter-derived time and the scope pulse agree at every level, as in Task 2.

A table of the four levels with the text size and the time per call at each. The predicted ratio and the measured ratio. One disassembly excerpt with the transformation named and the source lines it acts on. Your explanation of the trade-off between time and code size.

Question. State your measured time at `-O0` and at `-O2` in microseconds. State the ratio you measured and compare it to the ratio you predicted. Name the transformation you identified in the disassembly and the source lines in your own code on which it acted.

Task 4: Assembly Cycle Counting & Oscilloscope

Disconnect the POT0 jumper and connect the Lab Signal Generator to the PB0 pin on the 2x2 header. Configure the signal generator to output a 1 kHz sine wave with an amplitude of $3.0 V_{pp}$ (0 V to 3.0 V) and a DC offset of 1.5 V.

Write a pure Assembly loop that continuously reads the ADC on PB0 and immediately outputs the sampled value to DAC1 on PA4. A precise, cycle-counted software delay must be inserted into the polling loop such that the output sine wave on PA4 is delayed by exactly 45° ($\frac{\pi}{4}$ radians) relative to the input signal on PB0.

Deliverables:

1. The Assembly source code, heavily commented with the cycle-counting calculations. Assume that the system clock is running at the default 8 MHz internal RC oscillator.
2. A screenshot from the laboratory oscilloscope showing:
 - CH1 connected to PB0 (input signal).
 - CH2 connected to PA4 (output signal).
 - Oscilloscope cursors demonstrating a time difference of exactly $125 \mu s$ between the two signals.

This time difference corresponds to a 45° phase shift at a 1 kHz input frequency:

$$T = \frac{1}{f} = \frac{1}{1000} = 1 \text{ ms}$$
$$\Delta t = \frac{45^\circ}{360^\circ} T = \frac{1}{8} (1 \text{ ms}) = 125 \mu s.$$

Task 5: Assembly & Analog Debugging

Write an Assembly subroutine to initialize the LCD and display the letter “A” on the screen. The LCD must be controlled using bit-banging, with the following pins:

- Enable: PC15
- Register Select (RS): PC14
- Data pins: PA15, PA12, PB9, PB8

The LCD must be operated in 4-bit mode.

The initial implementation will likely fail to display anything. To investigate the problem, connect:

- CH1 of the oscilloscope to the 3.3 V side of the Enable signal (PC15).
- CH2 of the oscilloscope to the 5 V side of the Enable signal (PC15_S).

Deliverables:

1. Measure the time taken for PC15_S to rise from 0 V to 3.5 V, where 3.5 V is the logic-high threshold for the LCD. Provide an oscilloscope screenshot showing this measurement.
2. Use the standard capacitor-charging equation to calculate the actual parasitic capacitance of the Enable trace on the specific board:

$$V_C(t) = V_{CC} \left(1 - e^{-t/(RC)} \right).$$

Rearranging for capacitance gives:

$$C = \frac{-t}{R \ln \left(1 - \frac{V_C(t)}{V_{CC}} \right)}.$$

Use the measured rise time, the known resistance R , the supply voltage V_{CC} , and the measured threshold voltage $V_C(t) = 3.5$ V to determine the parasitic capacitance.

3. Modify the Assembly `Enable_Pulse` subroutine to include a sufficient number of `NOP` instructions such that the 5 V signal remains above 3.5 V for at least 450 ns, satisfying the HD44780 minimum Enable pulse-width requirement.
4. Provide the modified Assembly source code with the cycle-counting calculations showing how the required number of `NOP` instructions was determined.

3 Report Requirements

Format and Submission

- Submit one PDF. Do not submit a zip archive for the report itself. Write in L^AT_EX.
- Filename: `prac1B_studentnumber1_studentnumber2.pdf`. Upload to Gradescope before your demo and push your code to github, same repo you used for practical1A.

Required Structure

Write five numbered sections in the following order, followed by references and the code appendix.

Section 1: Golden Measure and PC Timing, Task 1 (2 pages)

Include:

- Predicted time per call in nanoseconds, including the instruction count estimate and CPU clock used for the prediction.
- Repetition count for each of the two runs.
- Measured time per call for each run, the mean, and the spread across runs.
- The complete output table for the ten inputs.
- One hand check written out in full.
- Your answer to the Task 1 question.

Section 2: Board Implementation and Timer Measurement, Task 2 (3 pages plus figures)

Include:

- The clock path you configured, naming each divider and giving the prescaler value you set.
- Predicted count span and predicted time, including the reasoning.
- Measured count span and the complete calculation from divider values to microseconds.
- Scope pulse width with the cursor screenshot.
- Wrap-around handling, including the long-run total, wrap arithmetic, and mean time per call.
- Pass or fail evidence against the ten golden values.
- Your answer to the Task 2 question.

Section 3: Optimisation Flags, Task 3 (2 pages plus figures)

Include:

- One table covering all four optimisation levels. Include the flag, text size in bytes, counter-derived time per call, scope pulse width, and ratio to the -O0 baseline.
- Predicted ratio and measured ratio, shown side by side.
- One disassembly excerpt with the transformation named and the source lines it affects.
- Your explanation of the trade-off between execution time and code size.
- Your answer to the Task 3 question.

Section 4: Cycle-Counted Phase Delay, Task 4 (2 pages plus figures)

Include:

- The Assembly listing, commented per instruction with the cycle cost of each instruction.
- A cycle budget table containing the instruction, cycles per instruction, iteration count, subtotal, and running total.
- The derivation of $125\ \mu\text{s}$ from the 8 MHz system clock and the 1 kHz input.
- The oscilloscope screenshot with CH1 connected to PB0, CH2 connected to PA4, and cursors placed on the measured time difference.
- The measured Δt , the error relative to $125\ \mu\text{s}$, and the source of the error.

Section 5: Analog Debugging of the LCD Enable Line, Task 5 (3 pages plus figures)

Include:

- The Assembly subroutine for 4-bit initialisation and displaying the letter “A”.
- The failing case, with a screenshot of PC15 and PC15_S together showing the 5 V side falling short of 3.5 V.
- The measured rise time from 0 V to 3.5 V on PC15_S, with cursors.
- The capacitance calculation, stating R , V_{CC} , and $V_C(t)$, using:

$$C = \frac{-t}{R \ln \left(1 - \frac{V_C}{V_{CC}} \right)}.$$

- The NOP count derivation, showing the measured rise time and the 450 ns HD44780 hold requirement converted to cycles at 125 ns per cycle.
- The fixed case, with a screenshot showing PC15_S above 3.5 V for at least 450 ns.
- A photograph of the LCD displaying “A”.

References

Include every manual section, datasheet table, and external source used in the report.

Oscilloscope Evidence

Oscilloscope screenshots provide the proof for every timing claim. A timing value without a matching oscilloscope screenshot receives zero for the measurement.

Submit a minimum of eight screenshots:

Task	Screenshots	What each shows
Task 2	1	Single-call pulse on PC13, cursors on falling and rising edge
Task 3	4	Pulse at -00, -01, -02, and -0s
Task 4	1	CH1 on PB0 and CH2 on PA4, cursors showing $125\ \mu\text{s}$
Task 5	2	PC15 and PC15_S before and after the fix

Table 1: Minimum oscilloscope evidence required in the report.

Every screenshot must show:

- The full oscilloscope screen, uncropped.
- Both traces with the channel numbers visible.

- Volts per division and time per division.
- Both cursors placed on the edges being measured, with the delta readout visible on screen.

Every figure must include:

- A figure number.
- A caption naming the pins probed, the quantity measured, and the value read.
- A reference from the body text using the figure number.
- A value in the body text matching the value shown on the screen.

Mismatched values are marked as unsupported.

Capture Rules

- Export screenshots over USB where the instrument supports this.
- Where no export option exists, photograph the oscilloscope screen square on and in focus, with all readouts legible.
- Do not annotate over the readouts. Add arrows and labels outside the trace area only.
- Use one capture for each measurement. Do not reuse a single screenshot for two different results.
- Screenshots from another group, a simulation, a previous year, or an online source receive zero for the task and are referred to the faculty for review.

3.1 No Video Submissions

Do not submit video files.

Do not submit links to video.

Videos will not be marked.

Live behaviour is assessed in person during the demonstration session in the White Lab.

Bring your board, build environment, and report to the demonstration session.

Be ready to:

- Rebuild at an optimisation level specified by the demonstrator and rerun the timed measurement.
- Place the oscilloscope cursors yourself while the demonstrator observes.
- Show the LCD displaying “A” and the phase-shifted DAC output on the oscilloscope.
- Answer questions about any line of your own code.

Both partners must answer questions.

A result reported in the report which you are unable to reproduce or explain at the bench will be withdrawn.